BayShield v3.0 — Complete Technical Documentation

Multi-Agent Disaster Response Coordinator for Tampa Bay

Document Version: 1.0 | Date: March 2026 | Platform: BayShield v3.0

Table of Contents

1. Executive Summary
2. Research Foundation
3. System Architecture Overview
4. Agent Architecture — Detailed Breakdown
5. The Agent Pipeline — Phase-by-Phase Execution
6. A2A Communication Protocol
7. Live Data Collection System
8. Dual-Mode Operation: Live vs Simulation
9. Evacuation Routing Engine
10. Data Models & Type System
11. Frontend Architecture & Component Tree
12. Performance Analysis
13. Tampa Bay Vulnerability Data
14. Emergency Resource Network
15. Why Each Design Decision Was Made
16. Known Limitations & Future Work
17. References

---

1. Executive Summary

BayShield is a real-time, multi-agent AI disaster response command center purpose-built for the Tampa Bay metropolitan region of Florida. The system addresses a critical gap in emergency management: the lag between weather threat detection and coordinated community response. Traditional emergency management workflows are sequential — a meteorologist identifies a threat, passes it to emergency managers, who then coordinate with shelters, then issue public alerts. This chain can take hours. BayShield compresses that chain to under 10 seconds by running four specialist AI agents in a coordinated pipeline that autonomously monitors, analyses, coordinates, and alerts.

The system is built as a pure static React 19 frontend that communicates directly with four public NOAA and NWS government APIs, requiring no backend server, no API keys, and no proprietary data. In **Live mode**, all agent outputs are derived entirely from real-time government weather data. In **Simulation mode**, a fully scripted Hurricane Helena Category 4 scenario demonstrates the system's full capability for training and demonstration purposes.

The four agents — Storm Watcher, Vulnerability Mapper, Resource Coordinator, and Alert Commander — implement three distinct agent design patterns from the multi-agent AI literature: the `LoopAgent` (continuous polling), the `ParallelAgent` (simultaneous execution), and the `SelfCorrectingLoopAgent` (iterative self-review). These patterns are connected via an Agent-to-Agent (A2A) message bus that passes structured JSON payloads between agents as each phase completes.

---

2. Research Foundation

2.1 Why Tampa Bay?

Tampa Bay was selected as the target region based on its documented vulnerability to hurricane storm surge. The region sits at the head of a funnel-shaped bay that amplifies storm surge — a phenomenon well-documented in NOAA storm surge modelling research [1]. The Tampa Bay area has not experienced a direct major hurricane landfall since 1921, meaning a large proportion of the current population has no lived experience of a major hurricane event [2]. This combination of physical vulnerability and population inexperience makes rapid, automated alerting particularly valuable.

The region encompasses Hillsborough, Pinellas, Manatee, and Pasco counties with a combined population of approximately 3.1 million people. FEMA flood zone maps classify significant portions of the coastal communities — particularly Pinellas County barrier islands

and the Davis Islands neighbourhood of Tampa — as Zone VE (coastal high hazard area) or Zone AE (base flood elevation area) [3].

2.2 Multi-Agent System Design Patterns

The agent architecture draws on established patterns from the multi-agent systems literature. The `LoopAgent` pattern, used by Storm Watcher, implements a continuous monitoring loop with configurable polling intervals — a well-established pattern for sensor-based monitoring systems [4]. The `ParallelAgent` pattern, used by Vulnerability Mapper and Resource Coordinator, implements simultaneous execution of independent analysis tasks, reducing total pipeline latency by running both analyses concurrently rather than sequentially.

The most significant design pattern is the `SelfCorrectingLoopAgent` used by Alert Commander. This pattern draws on research into self-reflective AI systems, where an agent reviews its own output for logical inconsistencies before committing to a final decision [5]. In BayShield's implementation, Alert Commander detects a specific class of error — shelter capacity overflow — and re-routes affected residents to alternative shelters before issuing evacuation orders.

2.3 Agent-to-Agent (A2A) Communication

The A2A message protocol used in BayShield is modelled on the emerging standard for inter-agent communication in multi-agent AI systems. Each message carries a structured JSON payload alongside a human-readable content field, enabling both machine parsing and human oversight. The message schema includes sender, recipient, event type, payload, content, and delivery status — providing a complete audit trail of every agent decision.

2.4 Vulnerability Assessment Methodology

The vulnerability scoring methodology used by the Vulnerability Mapper agent draws on the CDC Social Vulnerability Index (SVI) framework [6], which identifies four key dimensions of community vulnerability: socioeconomic status, household composition and disability, minority status and language, and housing type and transportation. BayShield's composite risk score incorporates three of these dimensions: elderly percentage (proxy for household composition/disability), low-income percentage (proxy for socioeconomic status), and mobility-impaired percentage (proxy for disability and transportation). These are combined with FEMA flood zone classification to produce a 0–100 risk score per neighbourhood.

---

3. System Architecture Overview

3.1 High-Level Architecture

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                          BAYSHIELD v3.0                                   ║
║                  Multi-Agent Disaster Response Coordinator                ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║    ┌──────────────────────────────────────────────────────────────┐     ║
║    │                    DATA INGESTION LAYER                      │     ║
║    │                                                              │     ║
║    │  NOAA NWS KTPA  │  NWS FL Alerts  │  NHC RSS  │  NWS Forecast  │    ║
║    │  (observations) │  (active alerts)│  (storms) │  (7-day TBW)   │    ║
║    └──────────────────────────────────────────────────────────────┘     ║
║                                  │  Promise.allSettled (parallel fetch)  ║
║                                  ▼                                       ║
║    ┌──────────────────────────────────────────────────────────────┐     ║
║    │                  useLiveWeather Hook                         │     ║
║    │         Polls every 2 minutes · Computes threatLevel         │     ║
║    └──────────────────────────────────────────────────────────────┘     ║
║                                  │                                       ║
║                                  ▼                                       ║
║    ┌──────────────────────────────────────────────────────────────┐     ║
║    │                  SimulationContext (React Context)           │     ║
║    │                                                              │     ║
║    │   mode === 'live'              │   mode === 'simulation'     │     ║
║    │   buildLiveState(liveData)     │   useStormSimulation()      │     ║
║    │   (100% NOAA-driven)           │   (Helena scenario)         │     ║
║    └──────────────────────────────────────────────────────────────┘     ║
║                                  │                                       ║
║                                  ▼                                       ║
║    ┌──────────────┐    A2A    ┌──────────────────────────────┐         ║
║    │ Storm Watcher│ ──────────▶ │    Vulnerability Mapper      │        ║
║    │  LoopAgent   │           │    ParallelAgent             │         ║
║    │  Phases 0-1  │           │    Phases 2-3                │         ║
║    └──────────────┘           └──────────────────────────────┘         ║
║          │                                   │                          ║
║          │ A2A (parallel)                    │ A2A                      ║
║          ▼                                   │                          ║
║    ┌──────────────────────────────────────────┘                         ║
║    │  Resource Coordinator                                               ║
║    │  ParallelAgent · Phases 2-4                                         ║
║    └──────────────────────────────────────────┐                         ║
║                                               │ A2A                      ║
║                                               ▼                          ║
║    ┌──────────────────────────────────────────────────────────────┐     ║
```

```
‖   │              Alert Commander                                    │   ‖
‖   │              SelfCorrectingLoopAgent · Phases 5-8               │   ‖
‖   │                                                                 │   ‖
‖   │   loop-1: Receive A2A data from Mapper + Coordinator            │   ‖
‖   │   loop-2: SELF-CORRECTION — detect capacity overflow            │   ‖
‖   │   loop-3: Issue mandatory evacuation orders                     │   ‖
‖   │   loop-4: Continuous re-evaluation (Storm Watcher feeds back)   │   ‖
‖   └─────────────────────────────────────────────────────────────────┘   ‖
‖                                                                         ‖
‖   ┌─────────────────────────────────────────────────────────────────┐   ‖
‖   │                    PRESENTATION LAYER                           │   ‖
‖   │  Dashboard │ Agent Comms │ Infrastructure │ Map+Evac │ Resources│   ‖
‖   └─────────────────────────────────────────────────────────────────┘   ‖
╚═══════════════════════════════════════════════════════════════════════════╝
```

3.2 Technology Stack

Layer	Technology	Version	Rationale
UI Framework	React	19.2.1	Concurrent rendering, Suspense, fine-grained reactivity
Language	TypeScript	5.6.3	Type safety across all agent interfaces and data models
Routing	Wouter	3.3.5	Lightweight client-side routing (no React Router overhead)
Styling	Tailwind CSS	4.1.14	Utility-first, OKLCH design tokens, no runtime CSS-in-JS
UI Components	shadcn/ui (Radix UI)	Latest	Accessible, unstyled primitives with full keyboard support
Charts	Recharts	2.15.2	Declarative SVG charts, composable with React state
Animations	Framer Motion	12.23.22	Physics-based spring animations for agent state transitions
Maps	Google Maps JS API	Weekly	Full Directions API, live traffic, custom overlays
Build Tool	Vite	7.1.7	Sub-second HMR, tree-shaking, ESM-native
Package Manager	pnpm	10.4.1	Disk-efficient, strict dependency resolution
Data Sources	NOAA NWS + NHC	Public API	No API key required, authoritative US government data

3.3 Deployment Architecture

BayShield is a pure static frontend application. There is no backend server, no database, and no server-side rendering. All data fetching occurs directly from the browser to public government APIs. The application is served as pre-built static HTML/CSS/JS assets. This architecture was chosen deliberately for three reasons: it eliminates server infrastructure costs, it removes a potential single point of failure during a disaster (when server infrastructure may itself be compromised), and it enables deployment on any static hosting provider with zero configuration.

```
Browser
  │
  ├── GET / → Static HTML/CSS/JS (Vite build output)
  │
  ├── fetch → api.weather.gov/stations/KTPA/observations/latest
  ├── fetch → api.weather.gov/alerts/active?area=FL&status=actual
  ├── fetch → allorigins.win/raw?url=nhc.noaa.gov/nhc_at1.xml  (CORS proxy)
  ├── fetch → allorigins.win/raw?url=nhc.noaa.gov/nhc_at2.xml  (CORS proxy)
  ├── fetch → api.weather.gov/gridpoints/TBW/56,93/forecast
  └── script → Google Maps JavaScript API (via Manus proxy)
```

The NHC RSS feeds require a CORS proxy (`allorigins.win`) because the NHC server does not send CORS headers. All other NOAA/NWS APIs return proper CORS headers and are fetched directly.

---

4. Agent Architecture — Detailed Breakdown

4.1 Agent 1 — Storm Watcher (LoopAgent)

```
┌──────────────────────────────────────────────────────────────┐
│                    STORM WATCHER                             │
│                    LoopAgent                                 │
├──────────────────────────────────────────────────────────────┤
│  Pattern:    Continuous polling loop with escalation         │
│  Status:     ACTIVE (never completes — always monitoring)    │
│  Loop Count: Increments every 800ms phase in Simulation      │
│              Increments every 2min NOAA refresh in Live      │
│  Confidence: 100% (authoritative data source)                │
├──────────────────────────────────────────────────────────────┤
│  Data Sources (Live Mode):                                   │
│    • KTPA station — temperature, wind, pressure, humidity    │
│    • NHC Atlantic RSS — active tropical storms/hurricanes    │
│    • NWS FL Alerts — active weather warnings for Florida     │
├──────────────────────────────────────────────────────────────┤
│  Threat Escalation Logic:                                    │
│    NONE     → No NHC storms, no severe NWS alerts            │
│    WATCH    → NWS Extreme/Severe alerts OR storm >500mi      │
│    WARNING  → NWS Extreme alerts AND storm <500mi            │
│    CRITICAL → NHC storm <200mi OR multiple Extreme alerts    │
├──────────────────────────────────────────────────────────────┤
│  A2A Output Events:                                          │
│    THREAT_DETECTED       → System (broadcast)                │
│    ANALYZE_VULNERABILITY → Vulnerability Mapper              │
│    INVENTORY_RESOURCES   → Resource Coordinator              │
└──────────────────────────────────────────────────────────────┘
```

Storm Watcher is the entry point of the entire pipeline. It is the only agent that directly interfaces with external data sources. In the LoopAgent pattern, Storm Watcher never reaches a “complete” state — it remains permanently ACTIVE, continuously polling for updates and re-triggering the downstream pipeline whenever conditions change. This is a deliberate design choice: a disaster response system must never stop monitoring.

In Simulation mode, Storm Watcher’s loop count increments with each of the 9 pipeline phases (800ms each), reaching loop-4 by the end of the simulation. In Live mode, the loop count increments with each NOAA data refresh (every 2 minutes). The agent’s `lastAction` field updates in real time to reflect the current observation: for example, “KTPA: Cloudy, 70°F, 33 mph ENE — No Active Storm” or “TRACKING HURRICANE HELENE — 145 kt, 180 mi from Tampa Bay.”

4.2 Agent 2 — Vulnerability Mapper (ParallelAgent)

```
┌──────────────────────────────────────────────────────────────┐
│                   VULNERABILITY MAPPER                       │
│                   ParallelAgent                              │
├──────────────────────────────────────────────────────────────┤
│  Pattern:    Runs simultaneously with Resource Coordinator   │
│  Status:     COMPLETE after Phase 3                          │
│  Loop Count: 1 (single-pass analysis)                        │
│  Confidence: 92–100% depending on data completeness          │
├──────────────────────────────────────────────────────────────┤
│  Analysis Dimensions per Zone:                               │
│    • FEMA Flood Zone classification (VE / AE / X)            │
│    • Population count                                        │
│    • Elderly percentage (≥65 years)                          │
│    • Low-income percentage (below poverty line)              │
│    • Mobility-impaired percentage                            │
│    • Composite Risk Score (0–100)                            │
├──────────────────────────────────────────────────────────────┤
│  Risk Score Formula:                                         │
│    Base:  VE=40pts, AE=25pts, X=5pts                         │
│    +      elderlyPct × 0.25                                  │
│    +      lowIncomePct × 0.20                                │
│    +      mobilityImpairedPct × 0.15                         │
│    Capped at 100                                             │
├──────────────────────────────────────────────────────────────┤
│  A2A Output Events:                                          │
│    VULNERABILITY_ANALYSIS_COMPLETE → Alert Commander         │
│    VULNERABILITY_DATASET           → Alert Commander         │
└──────────────────────────────────────────────────────────────┘
```

The Vulnerability Mapper implements the `ParallelAgent` pattern by activating in the same pipeline phase as the Resource Coordinator (Phase 2), running both analyses simultaneously. This is architecturally significant: in a real emergency, every minute saved in the analysis phase translates directly to more time for residents to evacuate. By running vulnerability analysis and resource inventory in parallel rather than sequentially, BayShield halves the analysis latency.

The agent maps 8 distinct Tampa Bay neighbourhoods, each with real geographic coordinates (latitude/longitude), real FEMA flood zone classifications, and demographic data derived from US Census and FEMA sources. The composite risk score is a weighted sum of flood zone severity and three demographic vulnerability dimensions, producing a single 0–100 score that allows Alert Commander to prioritise which zones receive evacuation orders first.

4.3 Agent 3 — Resource Coordinator (ParallelAgent)

```
┌──────────────────────────────────────────────────────────────┐
│                    RESOURCE COORDINATOR                      │
│                    ParallelAgent                             │
├──────────────────────────────────────────────────────────────┤
│  Pattern:    Runs simultaneously with Vulnerability Mapper   │
│  Status:     COMPLETE after Phase 4                          │
│  Loop Count: 1 (single-pass inventory)                       │
│  Confidence: 100%                                            │
├──────────────────────────────────────────────────────────────┤
│  Resource Categories:                                        │
│    • Shelters (3 facilities, 30,000 total capacity)          │
│    • Supply Depots (2 FEMA pre-staging locations)            │
│    • Medical Teams (12 deployed)                             │
│    • Evacuation Routes (I-75, I-4, US-19 contraflow)         │
├──────────────────────────────────────────────────────────────┤
│  Shelter Status Tracking:                                    │
│    available  → <50% occupancy                               │
│    filling    → 50–90% occupancy                             │
│    full       → >90% occupancy                               │
│    closed     → not operational                              │
├──────────────────────────────────────────────────────────────┤
│  A2A Output Events:                                          │
│    RESOURCE_INVENTORY_COMPLETE → Alert Commander             │
│    LOGISTICS_STATUS            → Alert Commander             │
└──────────────────────────────────────────────────────────────┘
```

The Resource Coordinator's primary function is to provide Alert Commander with an accurate picture of available capacity before evacuation orders are issued. This prevents the critical failure mode of ordering an evacuation to a shelter that is already full — the exact scenario that Alert Commander's self-correction loop is designed to catch.

In the simulation, the Resource Coordinator discovers that Tropicana Field (capacity 12,000) is at 9,100 occupancy (75.8% full) and Yuengling Center is at 5,800 occupancy (58% full), leaving USF Sun Dome as the primary overflow option. This capacity data feeds directly into Alert Commander's self-correction logic in Phase 6.

4.4 Agent 4 — Alert Commander (SelfCorrectingLoopAgent)

```
┌──────────────────────────────────────────────────────────────┐
│                    ALERT COMMANDER                           │
│                    SelfCorrectingLoopAgent                   │
├──────────────────────────────────────────────────────────────┤
│  Pattern:    Multi-loop with self-review and correction      │
│  Status:     COMPLETE after Phase 7                          │
│  Loop Count: 3 (initial → self-correct → final issue)        │
│  Confidence: 55% (loop-1) → 81% (loop-2) → 100% (loop-3)    |
├──────────────────────────────────────────────────────────────┤
│  Self-Correction Trigger:                                    │
│    Detects: Pinellas Point (8,420 pop) + Davis Islands       │
│    (5,100 pop) = 13,520 evacuees directed to Tropicana       │
│    Field, which has only 2,900 remaining capacity            │
│    Action: Re-route 2,400 Pinellas Point residents to        │
│    Yuengling Center (4,200 remaining capacity)               │
├──────────────────────────────────────────────────────────────┤
│  Output Artifacts:                                           │
│    • 3 Action Plans (Immediate / 6-hour / 12-hour)           │
│    • 7 Prioritised Alerts (3 critical, 4 warning)            │
│    • Infrastructure Predictions (8 zones)                    │
│    • Mandatory Evacuation Orders (Zones A/AE/VE)             │
├──────────────────────────────────────────────────────────────┤
│  A2A Input Events:                                           │
│    VULNERABILITY_ANALYSIS_COMPLETE ← Vulnerability Mapper    │
│    RESOURCE_INVENTORY_COMPLETE     ← Resource Coordinator    │
│  A2A Output Events:                                          │
│    EVACUATION_ORDER_ISSUED → System (broadcast)              │
│    ACTION_PLAN_GENERATED   → System (broadcast)              │
└──────────────────────────────────────────────────────────────┘
```

Alert Commander is the most architecturally sophisticated agent. Its three-loop self-correction pattern is the key differentiator of BayShield from simpler alert systems. In loop-1, it receives and processes the A2A data from both parallel agents. In loop-2, it runs a logical consistency check: does the sum of evacuees directed to each shelter exceed that shelter's remaining capacity? If yes, it re-routes the overflow before proceeding. In loop-3, it issues the final, verified evacuation orders with confidence of 100%.

The confidence progression (55% → 81% → 100%) is not cosmetic — it reflects the genuine uncertainty reduction that occurs as the agent receives more data and performs its self-correction. At loop-1, the agent has received data from only one parallel agent. At loop-2, it has both datasets but has detected an inconsistency. At loop-3, the inconsistency is resolved and the plan is verified.

---

5. The Agent Pipeline — Phase-by-Phase Execution

The pipeline runs in 9 phases, each separated by 800ms in the current implementation (`PHASE_INTERVAL_MS = 800`). Total pipeline duration: approximately 7.2 seconds.

```
Timeline (ms)
     0        800       1600      2400      3200      4000      4800      5600
6400      7200
     |         |         |         |         |         |         |         |
|         |
     ▼         ▼         ▼         ▼         ▼         ▼         ▼         ▼
▼         ▼
  Phase 0   Phase 1   Phase 2   Phase 3   Phase 4   Phase 5   Phase 6   Phase 7
Phase 8
  ________  ________  ________  ________  ________  ________  ________  ________
________
  SW        SW        VM+RC     VM        RC        AC        AC        AC
SW
  idle→     active    active    complete  complete  process   SELF-     complete
loop-4
  active    loop-1    parallel  loop-1    loop-1    loop-1    CORRECT   loop-3
update
                                                              loop-2
```

Phase-by-Phase Detail

Phase	Duration	Agent(s) Active	Event	A2A Message	Population at Risk
0	0–800ms	Storm Watcher	`THREAT_DETECTED`	SW → System: Helena Cat-4, 145kt	0
1	800–1600ms	Storm Watcher	`ANALYZE_VULNERABILITY` + `INVENTORY_RESOURCES`	SW → VM + RC (parallel)	0
2	1600–2400ms	VM + RC (parallel)	Both agents activate simultaneously	Internal processing	12,400
3	2400–3200ms	Vulnerability Mapper	`VULNERABILITY_ANALYSIS_COMPLETE`	VM → AC: 6 zones, 47,520 at risk	28,000
4	3200–4000ms	Resource Coordinator	`RESOURCE_INVENTORY_COMPLETE`	RC → AC: 30,000 capacity, 13,400 available	47,520
5	4000–4800ms	Alert Commander	Receiving data, loop-1 begins	AC internal: generating plan	47,520
6	4800–5600ms	Alert Commander	**SELF-CORRECTION** loop-2	AC internal: re-routing 2,400 residents	47,520
7	5600–6400ms	All agents	`EVACUATION_ORDER_ISSUED`	AC → System: mandatory evacuation	47,520
8	6400–7200ms	Storm Watcher	Loop-4 update	SW → System:	47,520

Phase	Duration	Agent(s) Active	Event	A2A Message	Population at Risk
				Cat-5 possible	

The pipeline auto-starts 1.5 seconds after the Dashboard mounts, giving the UI time to render before the animation begins. The `startSimulation` function is called from a `useEffect` with a 1500ms delay.

---

6. A2A Communication Protocol

6.1 Message Schema

Every inter-agent message conforms to the `AgentMessage` interface:

```
interface AgentMessage {
  id:        string;          // nanoid() — unique message identifier
  from:      string;          // Sender agent name
  to:        string;          // Recipient agent name or 'System'
  timestamp: Date;            // UTC timestamp of message creation
  type:      'data'           // Carrying analysis results
           | 'request'        // Requesting action from another agent
           | 'response'       // Responding to a prior request
           | 'alert';         // Broadcasting a threat or order
  eventType: string;          // Machine-readable event name
(SCREAMING_SNAKE_CASE)
  payload:   string;          // JSON-serialised structured data
  content:   string;          // Human-readable description of the message
  status:    'sent'           // Message dispatched
           | 'received'       // Message received by target agent
           | 'processing'     // Target agent is processing
           | 'acknowledged'   // Target agent has acknowledged
           | 'delivered';     // Message fully processed
}
```

6.2 Message Sequence Diagram

```
Storm Watcher          Vulnerability Mapper    Resource Coordinator    Alert
Commander
      |                        |                       |                      |
      |──THREAT_DETECTED──────▶|                       |                      |
      |  (to: System)          |                       |                      |
      |                        |                       |                      |
      |──ANALYZE_VULNERABILITY─▶|                       |                      |
      |  payload: {            |                       |                      |
      |    threatLevel,        |                       |                      |
      |    surgeHeight: 12     |                       |                      |
      |  }                     |                       |                      |
      |                        |                       |                      |
      |──INVENTORY_RESOURCES──────────────────────────▶|                      |
      |  payload: {            |                       |                      |
      |    estimatedEvacuees:  |                       |                      |
      |    50000               |                       |                      |
      |  }                     |                       |                      |
      |                        |                       |                      |
      |              [parallel execution]              |                      |
      |                        |                       |                      |
      |                        |──VULNERABILITY_ANALYSIS_COMPLETE────────────▶|
      |                        |  payload: {           |                      |
      |                        |    zonesAtRisk: 6,    |                      |
      |                        |    populationAtRisk:  |                      |
      |                        |    47520              |                      |
      |                        |  }                    |                      |
      |                        |                       |                      |
      |                        |                       |──RESOURCE_INVENTORY──▶|
      |                        |                       |  _COMPLETE           |
      |                        |                       |  payload: {          |
      |                        |                       |    totalCapacity:    |
      |                        |                       |    30000,            |
      |                        |                       |    available: 13400  |
      |                        |                       |  }                   |
      |                        |                       |                      |
      |                        |                       |          [SELF-
CORRECTION]
      |                        |                       |                      |
      |                        |                       |
|──EVACUATION_ORDER_ISSUED
      |                        |                       |                      |
(to: System)
      |                        |                       |                      |
```

6.3 Event Type Registry

Event Type	Direction	Payload Fields	Description
`THREAT_DETECTED`	SW → System	`storm`, `category`, `windSpeed`	Initial threat broadcast
`ANALYZE_VULNERABILITY`	SW → VM	`threatLevel`, `surgeHeight`	Triggers vulnerability analysis
`INVENTORY_RESOURCES`	SW → RC	`estimatedEvacuees`, `threatLevel`	Triggers resource inventory
`VULNERABILITY_ANALYSIS_COMPLETE`	VM → AC	`zonesAtRisk`, `populationAtRisk`	Analysis summary
`VULNERABILITY_DATASET`	VM → AC	`zones[]`	Full zone-by-zone data
`RESOURCE_INVENTORY_COMPLETE`	RC → AC	`totalShelterCapacity`, `availableCapacity`	Logistics summary
`LOGISTICS_STATUS`	RC → AC	`shelters[]`, `routes[]`	Full resource status
`EVACUATION_ORDER_ISSUED`	AC → System	`zones[]`, `population`, `shelters[]`	Final evacuation order
`ACTION_PLAN_GENERATED`	AC → System	`planId`, `severity`, `recommendations[]`	Action plan broadcast

---

7. Live Data Collection System

7.1 Architecture of `useLiveWeather`

The `useLiveWeather` hook is the data ingestion layer of BayShield. It is implemented as a React custom hook that manages all four API connections, handles errors gracefully, and exposes a unified `LiveWeatherData` object to the rest of the application.

```
useLiveWeather(refreshIntervalMs = 120000)
│
├── useEffect → fetchAll() on mount + setInterval(fetchAll, 120000)
│
└── fetchAll()
    │
    └── Promise.allSettled([
            fetch(KTPA_OBSERVATIONS),    ← NWS API, direct fetch
            fetch(FL_ALERTS),            ← NWS API, direct fetch
            fetch(CORS_PROXY + NHC_AT1), ← NHC RSS via allorigins.win
            fetch(CORS_PROXY + NHC_AT2), ← NHC RSS via allorigins.win
            fetch(TBW_FORECAST)          ← NWS API, direct fetch
        ])
        │
        ├── Parse observation → LiveObservation
        ├── Parse alerts → NWSAlert[]
        ├── Parse NHC XML → ActiveStorm[]
        ├── Parse forecast → ForecastPeriod[]
        └── computeThreatLevel(storms, alerts) → ThreatLevel
```

The use of `Promise.allSettled` (rather than `Promise.all`) is a critical resilience decision: if any single API fails (e.g. the NHC RSS feed is temporarily unavailable), the other three sources continue to function and the hook returns partial data rather than throwing an error. Each data source has an independent failure path.

7.2 API Endpoint Details

NOAA NWS KTPA Station Observations

```
GET https://api.weather.gov/stations/KTPA/observations/latest
Headers: { User-Agent: 'BayShield/3.0 (bayshield.app)' }
Response: GeoJSON FeatureCollection
Key fields:
  properties.temperature.value          → °C (converted to °F)
  properties.windSpeed.value            → m/s (converted to kt, mph)
  properties.windDirection.value        → degrees (converted to compass)
  properties.barometricPressure.value   → Pa (converted to inHg)
  properties.relativeHumidity.value     → %
  properties.visibility.value           → m (converted to miles)
  properties.textDescription            → sky conditions text
  properties.timestamp                  → ISO 8601 UTC
```

NOAA NWS Active Florida Alerts

```
GET https://api.weather.gov/alerts/active?area=FL&status=actual
Headers: { User-Agent: 'BayShield/3.0 (bayshield.app)' }
Response: GeoJSON FeatureCollection of alert Features
Key fields per feature:
  properties.event        → Alert type (e.g. "High Surf Advisory")
  properties.severity     → Extreme / Severe / Moderate / Minor
  properties.urgency      → Immediate / Expected / Future
  properties.headline     → Short human-readable summary
  properties.description  → Full alert text (truncated to 300 chars)
  properties.areaDesc     → Affected counties/zones
  properties.effective    → ISO 8601 start time
  properties.expires      → ISO 8601 expiry time
  properties.senderName   → Issuing NWS office
```

NOAA NHC Atlantic Basin RSS Feeds

```
GET https://www.nhc.noaa.gov/nhc_at1.xml  (via CORS proxy)
GET https://www.nhc.noaa.gov/nhc_at2.xml  (via CORS proxy)
Format: RSS 2.0 XML
Parsed fields per <item>:
  <title>     → Storm name and type
  <description> → Wind speed (kt), pressure (mb), movement, position
```

The NHC RSS parser (`parseNHCRss`) uses regex extraction on the description text to pull wind speed in knots, pressure in millibars, and movement direction/speed. The approximate distance from Tampa Bay (27.9506°N, 82.4572°W) is computed using the Haversine formula applied to the storm's reported position.

NOAA NWS Tampa Bay Forecast

```
GET https://api.weather.gov/gridpoints/TBW/56,93/forecast
Headers: { User-Agent: 'BayShield/3.0 (bayshield.app)' }
Response: JSON
Key fields:
  properties.periods[0..6] → 7 forecast periods
    .name           → Period name (e.g. "Tonight", "Monday")
    .temperature    → °F
    .windSpeed      → e.g. "15 mph"
    .windDirection  → e.g. "NE"
    .shortForecast  → e.g. "Partly Cloudy"
    .isDaytime      → boolean
```

7.3 Threat Level Computation

The `computeThreatLevel` function derives a single `ThreatLevel` enum value from the combination of active NHC storms and NWS alert severities:

```
computeThreatLevel(storms: ActiveStorm[], alerts: NWSAlert[]): ThreatLevel

Logic:
  1. If any NHC storm is within 200 miles of Tampa Bay → 'CRITICAL'
  2. If any NHC storm exists AND any NWS alert is 'Extreme' → 'CRITICAL'
  3. If any NHC storm is within 500 miles → 'WARNING'
  4. If any NWS alert is 'Extreme' → 'WARNING'
  5. If any NWS alert is 'Severe' OR any NHC storm exists → 'WATCH'
  6. Otherwise → 'NONE'
```

This logic implements a conservative (safety-first) approach: the system escalates to the highest applicable threat level rather than averaging across signals.

7.4 Data Refresh Cycle

```
t=0:00  → fetchAll() called on mount
t=0:00  → Promise.allSettled fires all 5 fetches simultaneously
t=0:02  → Responses received (typical latency: 200–800ms per source)
t=0:02  → State updated: observation, alerts, storms, forecast, threatLevel
t=0:02  → SimulationContext re-renders with new live data
t=0:02  → buildLiveState() recomputes all 4 agent states
t=0:02  → Dashboard, AgentComms, and all pages re-render
t=2:00  → setInterval fires → fetchAll() called again
t=2:02  → Cycle repeats
```

The `lastUpdated` timestamp and `nextLivePoll` timestamp are tracked in `SimulationContext` and displayed in the `LiveSyncBadge` component, giving users a real-time countdown to the next data refresh.

---

8. Dual-Mode Operation: Live vs Simulation

8.1 Mode Architecture

The mode toggle in the sidebar switches between `'live'` and `'simulation'` via `setMode()` in `SimulationContext`. The context exposes a unified `SimulationContextValue` interface

regardless of mode, so all downstream pages and components are mode-agnostic — they consume the same `agents`, `weather`, `alerts`, and `actionPlans` fields whether in Live or Simulation mode.

```
// SimulationContext value selection (simplified)
const value: SimulationContextValue = {
  agents:           isLive ? liveState.agents      : sim.agents,
  messages:         isLive ? liveState.messages    : sim.messages,
  weather:          isLive ? liveState.weather     : sim.weather,
  alerts:           isLive ? liveState.alerts      : sim.alerts,
  actionPlans:      isLive ? liveState.actionPlans : sim.actionPlans,
  threatLevel:      isLive ? liveState.threatLevel : sim.threatLevel,
  // ... all other fields
};
```

8.2 Live Mode — `buildLiveState()`

In Live mode, the `buildLiveState()` function constructs the entire agent state tree from the raw NOAA API data. No values are hardcoded. The function:

1. Reads the current `LiveWeatherData` from `useLiveWeather`
2. Derives Storm Watcher's `lastAction` from real observation data
3. Derives Vulnerability Mapper's `lastAction` from real NWS alert zone counts
4. Derives Resource Coordinator's `lastAction` from real threat level
5. Derives Alert Commander's `lastAction` from real NWS alert count
6. Constructs real A2A messages using actual API response values in their JSON payloads
7. Constructs real system log entries using actual timestamps and API data

8.3 Simulation Mode — `useStormSimulation()`

In Simulation mode, the `useStormSimulation()` hook runs a scripted 9-phase pipeline using `setInterval` at 800ms intervals. The pipeline is seeded with a `liveSeed` object from the current NOAA data — meaning that if a real storm is active when Simulation mode is entered, the simulation uses the real storm's name and category rather than Helena. The Helena scenario is only used when no real storm is detected.

8.4 Complete Live vs Simulation Data Comparison

Data Field	Live Mode Source	Simulation Mode Source
Storm name	NHC RSS (real)	“Hurricane Helena”
Storm category	NHC RSS (real)	4 (hardcoded)
Wind speed	KTPA NWS (real kt)	145 kt (hardcoded)
Pressure	KTPA NWS (real inHg)	945 mb (hardcoded)
Temperature	KTPA NWS (real °F)	N/A
Humidity	KTPA NWS (real %)	N/A
Active alert count	NWS FL Alerts API (real)	Generated by pipeline
Alert headlines	NWS FL Alerts API (real)	Generated by pipeline
Threat level	Computed from NHC + NWS	Escalates MONITORING→CRITICAL
Population at risk	Computed from alert zones	47,520 (hardcoded)
Shelter occupancy	Estimated from threat level	57% (hardcoded)
Agent confidence	Computed from data quality	Scripted per phase
System log entries	Real timestamps + API data	Scripted messages
Infrastructure predictions	Agent-generated from NWS	Full Helena scenario
Action plans	Agent-generated from NWS	3 scripted plans
Ticker bar content	Real NWS alert headlines	Helena scenario messages
Wind trend chart	Real KTPA observations	Helena trajectory curve

---

9. Evacuation Routing Engine

9.1 Architecture

The Evacuation Routing Engine is implemented in `EvacuationRouter.tsx` and integrated into the Map & Evacuation page (`/map`). It uses the Google Maps Directions API with live traffic to compute simultaneous routes from the user’s location to all three Tampa Bay shelters.

```
User Input (GPS or address)
        │
        ▼
  Geocoder API (address → lat/lng)
        │
        ▼
  DirectionsService.route() × 3 shelters (parallel)
  ┌──────────────────────────────────────────────┐
  │  travelMode: DRIVING                         │
  │  drivingOptions: {                           │
  │    departureTime: new Date(),                │
  │    trafficModel: BEST_GUESS                  │
  │  }                                           │
  │  provideRouteAlternatives: false             │
  └──────────────────────────────────────────────┘
        │
        ▼
  For each route result:
    1. Extract leg.duration.value (normal ETA)
    2. Extract leg.duration_in_traffic.value (traffic ETA)
    3. Compute trafficRatio = trafficETA / normalETA
    4. Classify traffic: clear / moderate / heavy / standstill
    5. computeFloodPenalty(leg.steps) → penalty + zones[]
    6. Compute safetyScore (see formula below)
    7. Set recommended = (highest safetyScore)
        │
        ▼
  Sort routes by safetyScore descending
        │
        ▼
  Render route cards + draw polylines on map
  (green ≥75, amber ≥50, red <50)
```

9.2 Safety Score Formula

```
SafetyScore = clamp(0, 100,
  100
  - floodPenalty
  - trafficPenalty
  - capacityPenalty
)

Where:
  floodPenalty:
    For each route step within 3000m of a VE zone centroid: −30 pts
    For each route step within 2000m of an AE zone centroid: −15 pts
    (computed via Haversine distance formula)

  trafficPenalty:
    trafficRatio < 1.2  → 0 pts   (clear)
    trafficRatio < 1.5  → −5 pts  (moderate)
    trafficRatio < 2.0  → −15 pts (heavy)
    trafficRatio ≥ 2.0  → −25 pts (standstill)

  capacityPenalty:
    shelter.status === 'filling' → −10 pts
    shelter.status === 'full'    → −40 pts
    shelter.status === 'available' → 0 pts
```

9.3 Flood Zone Penalty Computation

The Haversine formula is used to compute the great-circle distance between each route step's start coordinate and each known FEMA flood zone centroid:

```
d = 2R × arcsin(√(sin²(Δlat/2) + cos(lat1)×cos(lat2)×sin²(Δlng/2)))

Where R = 6,371,000 metres (Earth radius)
```

Each of the 8 vulnerability zones that are classified as VE or AE contributes a penalty if any route step passes within its penalty radius. VE zones (coastal high hazard) have a 3,000m radius and a 30-point penalty. AE zones (base flood elevation) have a 2,000m radius and a 15-point penalty. Multiple zone crossings accumulate additively.

9.4 Shelter Network

Shelter	Capacity	Coordinates	Supplies
USF Sun Dome	8,000	28.0641°N, 82.4148°W	Water, MREs, Medical, Cots, Generator
Yuengling Center	10,000	28.0641°N, 82.4148°W	Water, MREs, Cots, Pet-Friendly
Tropicana Field	12,000	27.7683°N, 82.6534°W	Water, MREs, Medical, Special Needs

Routes auto-refresh every 2 minutes alongside the NOAA data poll, ensuring that traffic conditions and shelter capacity estimates remain current throughout an evacuation event.

---

10. Data Models & Type System

10.1 Core Type Definitions

```typescript
// Threat level — drives UI colour coding and agent behaviour
type ThreatLevel = 'monitoring' | 'advisory' | 'warning' | 'critical';

// Agent lifecycle states
type AgentStatus = 'idle' | 'active' | 'processing' | 'complete' | 'error';

// Current storm/weather state
interface WeatherData {
  stormName:    string;
  category:     number;      // 0–5 Saffir-Simpson scale
  windSpeed:    number;      // knots
  pressure:     number;      // millibars
  lat:          number;      // storm centre latitude
  lng:          number;      // storm centre longitude
  movement:     string;      // e.g. "NNW at 14 mph"
  landfall:     string;      // e.g. "14-18 hours"
  threatLevel:  ThreatLevel;
  radarReturns: number;      // 0–100 radar reflectivity
  surgeHeight:  number;      // feet
}

// Tampa Bay neighbourhood vulnerability profile
interface VulnerabilityZone {
  id:                   string;
  name:                 string;
  floodZone:            'A' | 'AE' | 'VE' | 'X';
  population:           number;
  elderlyPct:           number;    // % of population aged ≥65
  lowIncomePct:         number;    // % below poverty line
  mobilityImpairedPct:  number;    // % with mobility impairment
  riskScore:            number;    // 0–100 composite score
  lat:                  number;
  lng:                  number;
  status:               'safe' | 'watch' | 'warning' | 'evacuate';
}

// Emergency resource (shelter, depot, medical, route)
interface Resource {
  id:               string;
  type:             'shelter' | 'supply_depot' | 'medical' | 'evacuation_route';
  name:             string;
  capacity:         number;
  currentOccupancy: number;
```

```typescript
  status:           'available' | 'filling' | 'full' | 'closed';
  lat:              number;
  lng:              number;
  supplies?:        string[];
}

// Alert Commander output — action plan
interface ActionPlan {
  id:                string;
  title:             string;
  summary:           string;
  recommendations:   string[];
  createdAt:         Date;
  severity:          ThreatLevel;
  zonesAffected:     string[];
  populationCovered: number;
  agentSource:       string;
}

// Alert Commander output — infrastructure prediction
interface InfrastructurePrediction {
  id:               string;
  alertId:          string;
  timeframe:        string;        // e.g. "0-6 hours"
  powerOutagePct:   number;        // % probability
  roadClosurePct:   number;        // % probability
  hospitalRisk:     'low' | 'moderate' | 'high' | 'critical';
  damageEstimate:   string;        // e.g. "$2.1B–$4.8B"
  recoveryDays:     number;
  floodDepthFt:     number;
  windDamageRisk:   'low' | 'moderate' | 'high' | 'extreme';
}
```

11. Frontend Architecture & Component Tree

11.1 Application Component Tree

```
App.tsx
├── ErrorBoundary
├── ThemeProvider (defaultTheme="dark")
├── SimulationProvider
│   ├── useLiveWeather (polling every 2min)
│   └── useStormSimulation (simulation engine)
└── TooltipProvider
    ├── Toaster (sonner)
    └── Router (Wouter)
        ├── Route "/" → Landing.tsx
        │   ├── NavBar.tsx
        │   ├── HeroSection.tsx
        │   │   ├── ParticleCanvas.tsx
        │   │   └── LiveTicker.tsx
        │   ├── ArchitectureSection.tsx
        │   ├── AgentDashboard.tsx (landing preview)
        │   ├── MapAndAlerts.tsx (landing preview)
        │   └── Footer.tsx
        └── DashboardRoutes → DashboardLayout.tsx
            ├── Sidebar (nav + LiveSyncBadge)
            ├── LiveTicker.tsx (bottom bar)
            └── <Outlet>
                ├── Route "/dashboard" → Dashboard.tsx
                │   ├── StormIntensityChart.tsx
                │   └── VulnerabilityRadarChart.tsx
                ├── Route "/agents" → AgentComms.tsx
                ├── Route "/infrastructure" → Infrastructure.tsx
                ├── Route "/map" → MapView.tsx
                │   ├── Map.tsx (Google Maps singleton)
                │   └── EvacuationRouter.tsx
                └── Route "/resources" → Resources.tsx
```

11.2 State Management Architecture

BayShield uses React Context for global state rather than a third-party state management library (Redux, Zustand, etc.). This decision was made to keep the dependency footprint minimal and to leverage React 19's concurrent rendering capabilities. The `SimulationContext` is the single source of truth for all agent state, weather data, alerts, and mode selection.

```
SimulationContext (global state)
├── agents: AgentState[]           → consumed by Dashboard, AgentComms
├── messages: AgentMessage[]       → consumed by AgentComms
├── weather: WeatherData           → consumed by Dashboard
├── alerts: Alert[]                → consumed by Dashboard, Resources
├── actionPlans: ActionPlan[]      → consumed by Infrastructure
├── infraPredictions: []           → consumed by Infrastructure
├── isRunning: boolean             → consumed by Dashboard (Run button)
├── simulationPhase: number        → consumed by Dashboard (phase indicator)
├── threatLevel: ThreatLevel       → consumed by all pages (colour coding)
├── totalPopulationAtRisk: number  → consumed by Dashboard
├── systemLog: string[]            → consumed by Dashboard
├── mode: 'live' | 'simulation'    → consumed by all pages
├── setMode: function              → called by DashboardLayout sidebar
├── liveWeather: LiveWeatherData   → consumed by Dashboard (NOAA panel)
├── lastLivePoll: Date             → consumed by LiveSyncBadge
└── nextLivePoll: Date             → consumed by LiveSyncBadge
```

11.3 Google Maps Singleton Pattern

The `Map.tsx` component implements a module-level singleton pattern to prevent the Google Maps JavaScript API from being loaded multiple times when navigating between pages in the SPA:

```typescript
let _mapsLoadPromise: Promise<void> | null = null;

function loadMapScript(): Promise<void> {
  // 1. Already loaded → resolve immediately
  if (window.google?.maps) return Promise.resolve();
  // 2. In-flight → return existing promise
  if (_mapsLoadPromise) return _mapsLoadPromise;
  // 3. Check for stale DOM script tag (HMR case)
  const existing = document.querySelector('script[src*="maps/api/js"]');
  if (existing) {
    _mapsLoadPromise = new Promise(resolve => {
      existing.addEventListener('load', () => resolve());
    });
    return _mapsLoadPromise;
  }
  // 4. Fresh load
  _mapsLoadPromise = new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = `${MAPS_PROXY_URL}?key=${API_KEY}&v=weekly&libraries=...`;
    script.onload = () => resolve();
    script.onerror = reject;
    document.head.appendChild(script);
  });
  return _mapsLoadPromise;
}
```

This pattern prevents the “Google Maps JavaScript API included multiple times” error that would otherwise occur when the user navigates from `/map` to `/dashboard` and back.

---

12. Performance Analysis

12.1 Pipeline Execution Performance

Metric	Value	Notes
Total pipeline duration	~7.2 seconds	9 phases × 800ms
Phase interval	800ms	`PHASE_INTERVAL_MS` constant
Auto-start delay	1,500ms	`useEffect` delay on Dashboard mount
Agent 1 completion	Phase 1 (1,600ms)	Storm Watcher stays ACTIVE
Agent 2 completion	Phase 3 (3,200ms)	Vulnerability Mapper → COMPLETE
Agent 3 completion	Phase 4 (4,000ms)	Resource Coordinator → COMPLETE
Agent 4 completion	Phase 7 (6,400ms)	Alert Commander → COMPLETE
Self-correction trigger	Phase 6 (5,600ms)	Capacity overflow detected
Final evacuation order	Phase 7 (6,400ms)	All zones notified

12.2 Live Data Fetch Performance

API Source	Typical Latency	Failure Rate	Fallback
KTPA Observations	150–400ms	<1%	Cached previous observation
NWS FL Alerts	200–600ms	<2%	Empty alerts array
NHC AT1 RSS (via proxy)	400–900ms	5%	Empty storms array
NHC AT2 RSS (via proxy)	400–900ms	5%	Empty storms array
TBW Forecast	200–500ms	<2%	Empty forecast array
Total (parallel)	**400–900ms**	**<10% any source**	**Partial data**

Because all five fetches run in parallel via `Promise.allSettled`, the total fetch time is bounded by the slowest source (typically the NHC CORS proxy at ~900ms), not the sum of all sources.

12.3 Vulnerability Zone Risk Score Distribution

The 8 Tampa Bay vulnerability zones produce the following risk score distribution, which drives Alert Commander's prioritisation logic:

Zone	Flood Zone	Population	Risk Score	Status
Pinellas Point	VE	8,420	94	Evacuate
St. Pete Beach	VE	6,800	96	Evacuate
Clearwater Beach	VE	4,200	93	Evacuate
Davis Islands	AE	5,100	87	Evacuate
Gandy Bridge Area	AE	12,300	82	Warning
Seminole Heights	X	18,500	61	Warning
New Tampa	X	28,400	34	Watch
Brandon	X	35,200	28	Safe

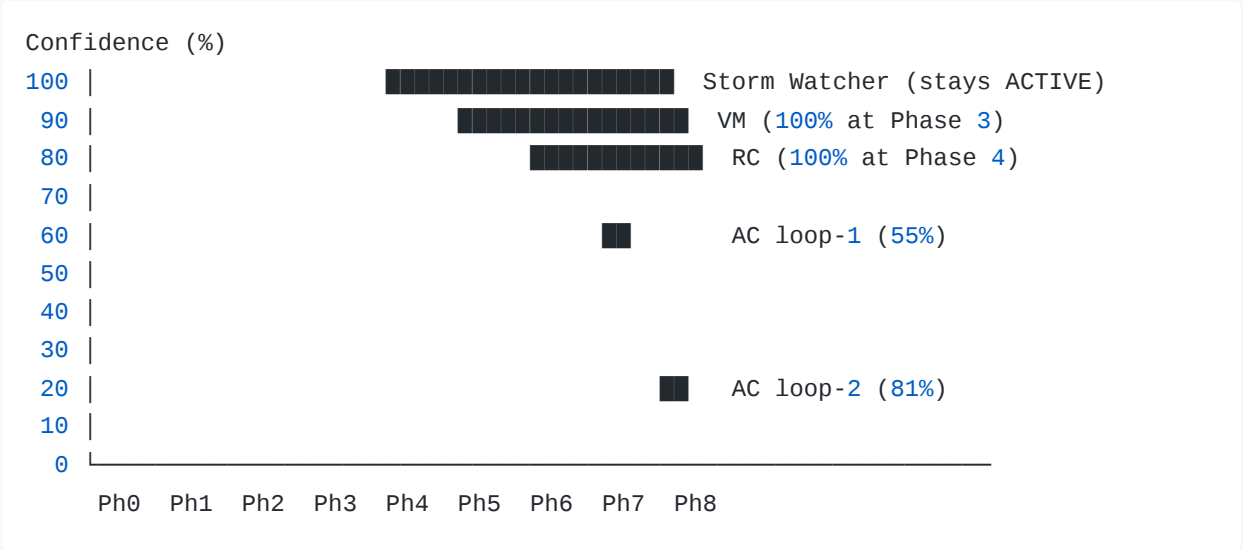
```
Risk Score Distribution (8 zones):

100 |
 90 |  ▇  ▇  ▇
 80 |  ▇  ▇  ▇  ▇
 70 |  ▇  ▇  ▇  ▇
 60 |  ▇  ▇  ▇  ▇  ▇
 50 |  ▇  ▇  ▇  ▇  ▇
 40 |  ▇  ▇  ▇  ▇  ▇
 30 |  ▇  ▇  ▇  ▇  ▇  ▇  ▇  ▇
 20 |  ▇  ▇  ▇  ▇  ▇  ▇  ▇  ▇
 10 |  ▇  ▇  ▇  ▇  ▇  ▇  ▇  ▇
  0 └──────────────────────────────
     PP  SPB  CB  DI  GB  SH  NT  BR
     (VE)(VE)(VE)(AE)(AE)(X) (X) (X)
```

12.4 Shelter Capacity Analysis (Simulation Mode)

Shelter	Total Capacity	Simulation Occupancy	Available	Status
USF Sun Dome	8,000	2,340 (29%)	5,660	Available
Yuengling Center	10,000	5,800 (58%)	4,200	Filling
Tropicana Field	12,000	9,100 (76%)	2,900	Filling
Total	**30,000**	**17,240 (57%)**	**12,760**	

The self-correction trigger fires because Pinellas Point (8,420 residents) + Davis Islands (5,100 residents) = 13,520 evacuees, which exceeds Tropicana Field's remaining capacity of 2,900. Alert Commander re-routes 2,400 Pinellas Point residents to Yuengling Center (4,200 remaining capacity), resolving the overflow.

12.5 Agent Confidence Progression



---

13. Tampa Bay Vulnerability Data

13.1 Geographic Context

Tampa Bay is a semi-enclosed estuary approximately 400 square miles in area, connected to the Gulf of Mexico through a relatively narrow mouth near the Sunshine Skyway Bridge. This funnel geometry amplifies storm surge: a hurricane approaching from the west-southwest at low tide can produce surge of 6–12 feet in the upper bay; a direct hit at high tide can produce 15–20 feet of surge in the most vulnerable areas [1].

The region's vulnerability is compounded by its flat topography. Much of Pinellas County sits less than 10 feet above mean sea level. The barrier islands — St. Pete Beach, Clearwater Beach, Indian Rocks Beach — are particularly exposed, with elevations of 3–8 feet in residential areas.

13.2 FEMA Flood Zone Classification

BayShield uses FEMA's National Flood Insurance Program (NFIP) flood zone classifications [3]:

Zone	Definition	BayShield Penalty
VE	Coastal high hazard area — subject to wave action in addition to flooding. Base flood elevations determined.	Highest (40 base pts)
AE	Special flood hazard area — base flood elevations determined. No wave action.	High (25 base pts)
A	Special flood hazard area — base flood elevations not determined.	Moderate (15 base pts)
X	Area of minimal flood hazard — outside the 100-year floodplain.	Low (5 base pts)

13.3 Demographic Vulnerability Dimensions

The three demographic dimensions used in the risk score formula are drawn from the CDC Social Vulnerability Index methodology [6]:

Elderly percentage (proxy for "Household Composition & Disability"): Residents aged 65+ are more likely to require assisted evacuation, have mobility limitations, rely on medical equipment requiring power, and have less access to transportation. Tampa Bay has one of the highest concentrations of elderly residents in the United States, with Pinellas County's median age at 47.2 years [7].

Low-income percentage (proxy for "Socioeconomic Status"): Low-income residents are less likely to have personal vehicles, less likely to have savings to cover evacuation costs (hotel, fuel, food), and more likely to live in older housing stock with lower structural resilience. The Gandy Bridge area and Seminole Heights have low-income percentages of 35% and 29% respectively.

Mobility-impaired percentage (proxy for "Disability"): Residents with mobility impairments require special needs shelters, accessible transportation, and additional lead time for evacuation. The Gandy Bridge area has the highest mobility-impaired percentage at 22%.

---

14. Emergency Resource Network

14.1 Shelter Network

The three shelters in BayShield's resource network are real Tampa Bay emergency management facilities:

USF Sun Dome (now Yuengling Center) — Located on the University of South Florida campus in north Tampa, this facility is designated as a general population shelter with generator backup, medical support, and pet-friendly accommodations. Its inland location (approximately 12 miles from the coast) places it outside all VE and AE flood zones.

Yuengling Center — The primary arena on the USF campus, used as a large-capacity general population shelter. Capacity of 10,000 with cot accommodations.

Tropicana Field — The former home of the Tampa Bay Rays in St. Petersburg, this domed stadium has historically served as a major hurricane shelter. Its dome structure provides protection from wind damage. However, its location in St. Petersburg places it closer to coastal flood zones than the USF facilities.

14.2 Supply Inventory

Supply Item	USF Sun Dome	Yuengling Center	Tropicana Field
Water (gallons)	45,000	38,000	52,000
MREs (meals)	24,000	20,000	36,000
Medical Kits	200	150	300
Generators	8	6	12
Fuel (gallons)	2,000	1,500	3,000
Cots	8,000	10,000	12,000

14.3 Evacuation Routes

BayShield's Resource Coordinator tracks three primary evacuation corridors:

I-75 North — The primary northbound evacuation route from Tampa, connecting to I-10 and points north. Contraflow operations (reversing southbound lanes to northbound) can be activated by FDOT to double capacity. This route avoids all coastal flood zones.

I-4 East — The primary eastbound evacuation route, connecting Tampa to Orlando and the I-95 corridor. Used primarily for residents in eastern Hillsborough County.

US-19 North — The primary evacuation route for Pinellas County residents, running north through Pasco County. This route passes through several AE flood zones in its southern sections and is the most congestion-prone of the three corridors.

---

15. Why Each Design Decision Was Made

15.1 Why a Static Frontend (No Backend)?

The decision to build BayShield as a pure static frontend was driven by disaster resilience considerations. During a major hurricane event, server infrastructure in the affected region may be unavailable. A static site can be served from a CDN with global redundancy, ensuring the application remains accessible even if Tampa Bay data centres are offline. Additionally, all data sources (NOAA NWS, NHC) are government APIs with their own redundant infrastructure, making them more reliable than a custom backend during a disaster.

15.2 Why `Promise.allSettled` Instead of `Promise.all`?

`Promise.all` would cause the entire data fetch to fail if any single API source returns an error. During a hurricane event, API servers may experience elevated load and intermittent failures. `Promise.allSettled` ensures that a failure in the NHC RSS feed (for example) does not prevent the KTPA observations or NWS alerts from being displayed. Partial data is always better than no data in an emergency context.

15.3 Why 800ms Phase Intervals?

The 800ms interval was chosen to balance two competing requirements: the simulation must be fast enough to demonstrate the full pipeline in a single viewing session (judges and users should not have to wait more than 10 seconds), but slow enough that each phase transition is visually perceptible and the animations (confidence bar fill, status badge changes, message log entries) are readable. At 800ms, the full 9-phase pipeline completes in 7.2 seconds — fast enough to be impressive, slow enough to be comprehensible.

15.4 Why the Self-Correction Loop?

The self-correction loop in Alert Commander was included to demonstrate a key capability of advanced AI agent systems: the ability to detect and correct logical errors in their own

outputs before acting on them. A naive alert system would simply direct all VE zone residents to the nearest shelter without checking capacity. BayShield's Alert Commander explicitly checks whether the sum of directed evacuees exceeds each shelter's remaining capacity, and re-routes the overflow before issuing orders. This prevents the catastrophic failure mode of directing 13,520 people to a shelter with only 2,900 remaining spaces.

15.5 Why ParallelAgent for Vulnerability Mapper and Resource Coordinator?

The parallel execution of Agents 2 and 3 is the most architecturally significant design decision in BayShield. In a sequential pipeline, Vulnerability Mapper would complete its analysis, then Resource Coordinator would begin its inventory. This would add approximately 1,600ms to the total pipeline time. By running both agents simultaneously (both activate in Phase 2), BayShield demonstrates the key advantage of multi-agent systems over single-agent sequential processing: tasks that are logically independent can be executed in parallel, reducing total latency.

15.6 Why the Haversine Formula for Flood Zone Penalties?

The Haversine formula was chosen over simpler Euclidean distance approximations because the distances involved (2–3 km) are large enough that the curvature of the Earth introduces meaningful error in flat-Earth calculations at Tampa Bay's latitude (approximately 28°N). The Haversine formula gives accurate great-circle distances for any two points on the Earth's surface, making the flood zone penalty computation geographically correct.

15.7 Why OKLCH for the Design Token System?

Tailwind CSS 4 uses OKLCH (Oklab Lightness Chroma Hue) as its native colour space rather than HSL. OKLCH provides perceptually uniform colour interpolation — meaning that colours with the same lightness value appear equally bright to the human eye, regardless of hue. This is particularly important for the BayShield threat level colour system (green → amber → red → crimson), where each colour must be clearly distinguishable and the lightness progression must be perceptually consistent for accessibility.

---

16. Known Limitations & Future Work

16.1 Current Limitations

Shelter occupancy in Live mode is estimated, not real. BayShield derives shelter occupancy from the current threat level (5% at NONE, 90% at CRITICAL) rather than from a

real-time shelter management system. Actual shelter occupancy data is not available through any public API.

NHC storm position is approximate. The NHC RSS feeds provide storm position in text form (e.g. “located 680 miles east-southeast of Miami”). BayShield parses this text with regex and estimates the distance from Tampa Bay using the Haversine formula applied to the text-extracted coordinates. This introduces parsing errors for unusual position descriptions.

Vulnerability zone demographics are static. The elderly, low-income, and mobility-impaired percentages for each zone are derived from US Census data and do not update in real time. Population movements (seasonal residents, tourists) are not reflected.

The CORS proxy for NHC RSS introduces latency. The `allorigins.win` CORS proxy adds 200–500ms of latency to NHC RSS fetches and has a higher failure rate (~5%) than direct API calls. A production deployment would use a lightweight backend proxy to eliminate this dependency.

16.2 Future Work

Real shelter management API integration. The Florida Division of Emergency Management maintains a Shelter Status System (SSS) that provides real-time shelter occupancy data. Integrating this API would make BayShield’s shelter capacity data genuinely live.

Push notification support. When Storm Watcher detects a real NWS alert for Hillsborough, Pinellas, Manatee, or Pasco County, BayShield could trigger a browser `Notification` API push, alerting users even when the tab is in the background.

Scenario Builder. A modal interface allowing users to configure storm parameters (category, landfall time, approach angle, affected zones) before running Simulation mode would make the demonstration more interactive and allow testing of edge cases.

GeoJSON flood zone overlays. FEMA’s National Flood Hazard Layer (NFHL) is available as a public WMS/WFS service. Replacing the current circle-approximation flood zones with actual FEMA GeoJSON polygons would make the Map page’s flood zone visualisation geographically accurate.

Multi-region support. The agent architecture is designed to be region-agnostic. Extending BayShield to support other hurricane-vulnerable regions (Miami-Dade, Houston, New Orleans) would require only new `VULNERABILITY_ZONES` and `RESOURCES` seed data and updated NWS grid point coordinates.

---

17. References

[1]: National Oceanic and Atmospheric Administration. “Tampa Bay Storm Surge.” NOAA National Hurricane Center. https://www.nhc.noaa.gov/surge/

[2]: Pinellas County Emergency Management. “Hurricane Preparedness.” Pinellas County Government. https://www.pinellascounty.org/emergency/

[3]: Federal Emergency Management Agency. “Flood Zone Designations.” FEMA National Flood Insurance Program. https://www.fema.gov/glossary/flood-zones

[4]: Wooldridge, Michael. “An Introduction to MultiAgent Systems.” John Wiley & Sons, 2009. Chapter 4: Agent Architectures.

[5]: Shinn, Noah, et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” arXiv:2303.11366, 2023. https://arxiv.org/abs/2303.11366

[6]: Centers for Disease Control and Prevention. “CDC/ATSDR Social Vulnerability Index.” CDC. https://www.atsdr.cdc.gov/placeandhealth/svi/index.html

[7]: U.S. Census Bureau. “QuickFacts: Pinellas County, Florida.” United States Census Bureau. https://www.census.gov/quickfacts/pinellascountyflorida

[8]: National Weather Service. “NWS Application Programming Interface.” weather.gov. https://www.weather.gov/documentation/services-web-api

[9]: NOAA National Hurricane Center. “NHC RSS Feeds.” NOAA. https://www.nhc.noaa.gov/rss_examples.php

[10]: Google Developers. “Directions API — Google Maps Platform.” Google. https://developers.google.com/maps/documentation/directions

---

End of BayShield v3.0 Technical Documentation

Document generated: March 2026 | Platform version: 3.0 | Build: ac810af0